

**National University of Computer and Emerging Sciences  
Islamabad Campus**

---



PDC Spring 2026

# Semester Project Report

## CS-3006

Parallel Log File Analysis using MPI in C

Eman Akbar 22i-1588

Huda Imran 22i-1713

CY-A

Submitted on: 04/04/2026

---

Dataset: UNSW-NB15 Network Intrusion Dataset  
Environment: 2-node Kali Linux cluster (master + slave)  
MPI Implementation: MPICH

## Contents

|                                                           |    |
|-----------------------------------------------------------|----|
| 1. Introduction and Project Overview .....                | 3  |
| 2. Dataset : UNSW-NB15 .....                              | 3  |
| Attack categories targeted in this project .....          | 3  |
| Detection results (verified against serial baseline)..... | 3  |
| 3. Q1 : Parallel Detection of Malicious Activities .....  | 3  |
| 3.1 Parallelization Strategy.....                         | 3  |
| Batch and scatter workflow .....                          | 4  |
| 3.2 MPI Collectives Used in Q1 .....                      | 4  |
| 3.3 Key Technical Fixes Implemented .....                 | 4  |
| Checksum mismatch bug (np=3 root cause).....              | 4  |
| MPI_Allreduce correctness fix.....                        | 4  |
| File descriptor fix .....                                 | 4  |
| Buffer safety .....                                       | 4  |
| 3.4 Execution Log Per-Process Breakdown.....              | 5  |
| 4. Q2 : Cross-Process Correlation and Error Checking..... | 7  |
| 4.1 Distributed Attack Detection .....                    | 7  |
| Why MPI_Gatherv instead of MPI_Gather .....               | 7  |
| Cross-check algorithm.....                                | 7  |
| 4.2 Destination Port Scanning Detection .....             | 8  |
| Results with 4 processes .....                            | 8  |
| 4.3 MPI Collectives Used in Q2.....                       | 8  |
| 4.4 Checksum Validation .....                             | 8  |
| 4.5 Detailed Execution Log .....                          | 8  |
| 5. Q3 : Performance Analysis .....                        | 13 |
| 5.1 Experimental Setup.....                               | 13 |
| 5.2 Q1 Performance Results .....                          | 13 |
| 5.3 Q2 Performance Results .....                          | 13 |
| 5.4 Analysis of Results .....                             | 14 |
| Why MPI with 1 process is slower than serial .....        | 14 |
| Why 4 processes is the optimal point .....                | 14 |
| Why 8 processes causes severe degradation .....           | 14 |
| Communication overhead boundary data exchanges.....       | 14 |
| Load imbalance observation .....                          | 14 |
| Detection coverage vs process count (Q2) .....            | 14 |
| 5.5 Speedup Calculation Amdahl's Law Context.....         | 14 |
| 6. PDC Concepts Applied .....                             | 15 |
| 6.1 Message Passing Paradigm .....                        | 15 |
| 6.2 Master-Worker (Manager-Worker) Pattern.....           | 15 |
| 6.3 Data Decomposition.....                               | 15 |
| 6.4 Collective Communication Operations.....              | 15 |
| 6.5 Synchronization and Correctness .....                 | 16 |
| 6.6 Scalability and Amdahl's Law .....                    | 16 |
| 6.7 Load Balancing .....                                  | 16 |
| 6.8 Latency vs Throughput Trade-offs .....                | 16 |
| 7. Conclusions .....                                      | 16 |

## 1. Introduction and Project Overview

This project implements a fully parallel log file analysis system using the Message Passing Interface (MPI) standard in C. The system processes the UNSW-NB15 network intrusion dataset – a large real-world dataset containing over 2.5 million network connection records spread across four CSV files – to detect malicious activities including Denial of Service (DoS) attacks, Backdoor intrusions, and Reconnaissance activities.

The project is divided into three questions. Q1 implements the core parallel detection engine. Q2 extends this with cross-process correlation to identify distributed attack patterns across multiple IP addresses and destination ports. Q3 provides a quantitative performance analysis comparing the parallel implementation against a serial baseline across different process counts on a two-machine cluster.

## 2. Dataset : UNSW-NB15

The UNSW-NB15 dataset was created by the Australian Centre for Cyber Security (ACCS) and contains 2,540,044 network connection records across four CSV files (UNSW-NB15\_1.csv through UNSW-NB15\_4.csv). Each record has 49 features including source IP, destination IP, destination port, protocol type, and an attack category label in column 47 (0-indexed).

### Attack categories targeted in this project

- DoS (Denial of Service) – flooding attacks that exhaust system resources
- Backdoor – covert channels allowing unauthorised remote access
- Reconnaissance – network scanning and information gathering activities

### Detection results (verified against serial baseline)

| Attack Type     | Count          | % of Malicious     |
|-----------------|----------------|--------------------|
| DoS             | 1,167          | 33.7%              |
| Reconnaissance  | 1,759          | 50.8%              |
| Backdoor        | 534            | 15.4%              |
| Total malicious | 3,460          | 100%               |
| Global checksum | 29,833,185,496 | Integrity verified |

## 3. Q1 : Parallel Detection of Malicious Activities

Q1 implements the parallel log file analysis engine. The fundamental challenge is that the dataset is too large to load into memory at once, and sequential processing is slow. The solution uses MPI to split the work across multiple processes using a batch-and-scatter strategy.

### 3.1 Parallelization Strategy

The dataset is processed in batches of 20,000 rows at a time. For each batch, rank 0 (the master process) reads the rows from disk, then distributes them across all available processes using MPI\_Scatter. Each process independently scans its assigned rows for malicious patterns, and the results are aggregated back to rank 0 using MPI collectives.

## Batch and scatter workflow

- Rank 0 reads up to 20,000 rows from the current CSV file into a shared buffer
- MPI\_Bcast broadcasts the batch count so all ranks know whether to continue or break
- Chunk size is computed using ceiling division:  $\text{chunk} = (\text{batch\_count} + \text{size} - 1) / \text{size}$
- Padding rows beyond the real data are zeroed with memset() to prevent stale data corruption
- MPI\_Scatter distributes chunk \* MAX\_LINE bytes to each process
- Each process scans its chunk independently no inter-process communication during analysis

## 3.2 MPI Collectives Used in Q1

| Collective    | Purpose                                                                                    | Where used                  |
|---------------|--------------------------------------------------------------------------------------------|-----------------------------|
| MPI_Bcast     | Broadcast batch_count to all ranks each iteration; broadcast global_total before Allreduce | Every batch iteration       |
| MPI_Scatter   | Distribute log rows evenly from rank 0 to all processes                                    | Every batch iteration       |
| MPI_Gather    | Collect per-process DoS, Backdoor, Recon counts into arrays on rank 0 for logging          | After analysis, every batch |
| MPI_Reduce    | Aggregate the checksum (long long) from all processes to rank 0                            | After analysis, every batch |
| MPI_Allreduce | All ranks raise a distributed attack flag once global_total is known                       | After all files processed   |
| MPI_Wtime     | Measure wall-clock execution time from MPI_Init to final output                            | Start and end of main()     |

## 3.3 Key Technical Fixes Implemented

### Checksum mismatch bug (np=3 root cause)

The original code padded unused batch slots with only `batch[i][0] = '\0'`, leaving the remainder of each row with stale data from previous batches. After MPI\_Scatter, worker processes received these rows with non-null-terminated garbage, causing `strlen()` to return incorrect lengths and the checksum to accumulate wrong values. The fix replaces the single-byte null with `memset(batch[i], 0, MAX_LINE)` which zeroes the entire row.

### MPI\_Allreduce correctness fix

`global_total` was computed only on rank 0 since it accumulates from MPI\_Reduce results. All other ranks had `global_total = 0`, making `local_flag` always 0 on non-root ranks and the Allreduce meaningless. The fix broadcasts `global_total` from rank 0 first using MPI\_Bcast, then every rank raises a valid `local_flag` before Allreduce genuinely aggregates across all processes.

### File descriptor fix

All ranks were calling `fopen()` on every CSV file, wasting file descriptors and risking inconsistent state on slave machines. Fixed so only rank 0 opens and closes files; all other ranks simply receive data via MPI\_Scatter.

### Buffer safety

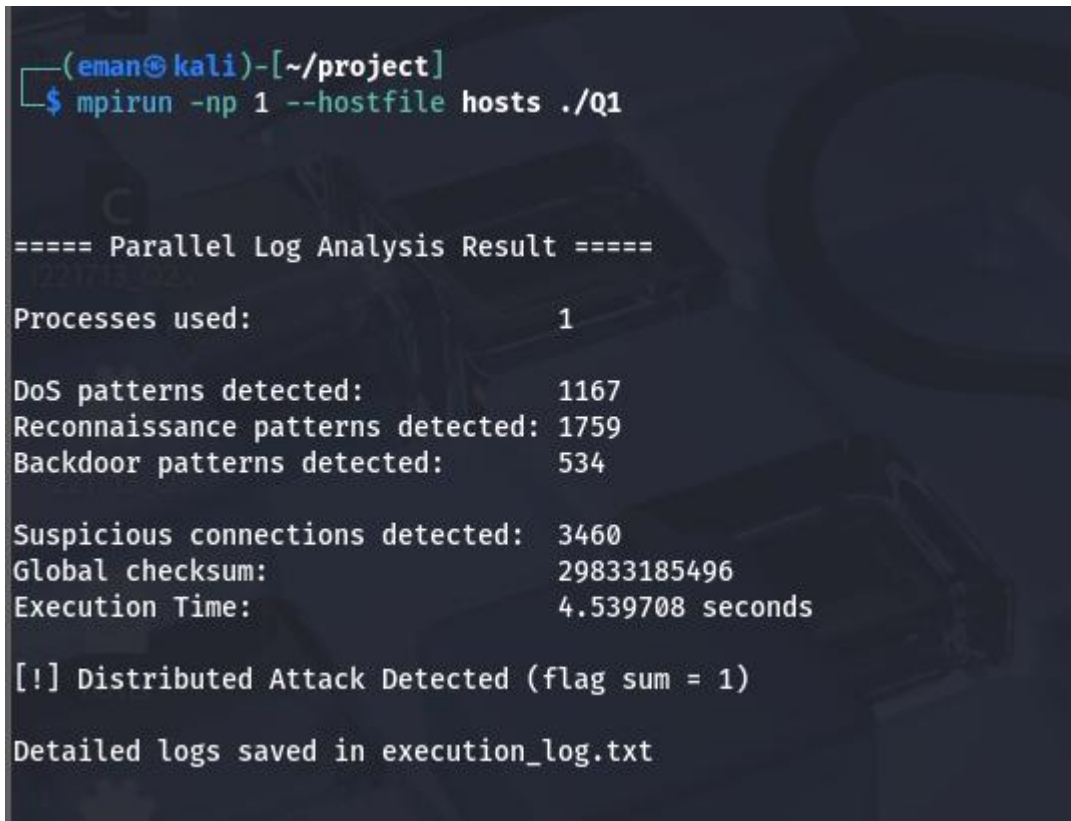
All `strcpy` calls replaced with `strncpy` with explicit length bounds to prevent buffer overflow. The checksum function casts each byte to unsigned char before accumulation to prevent sign-extension on bytes above 127, ensuring platform-consistent results.

### 3.4 Execution Log Per-Process Breakdown

The execution log (execution\_log.txt) records the per-process contribution for every batch via MPI\_Gather, for example from batch 5 of file 1:

```
process 0 - dos:271 backdoor:262 recon:34
process 1 - dos:211 backdoor:202 recon:29
process 2 - dos:25 backdoor:1 recon:46
process 3 - dos:20 backdoor:1 recon:45
```

This demonstrates load imbalance processes 0 and 1 received the rows containing the concentrated DoS/Backdoor cluster while processes 2 and 3 received clean traffic. This is an inherent consequence of MPI\_Scatter's sequential row distribution and is a key discussion point in the performance analysis.

A terminal window with a dark background and light blue text. The prompt is (eman@kali)-[~/project]. The command executed is \$ mpirun -np 1 --hostfile hosts ./Q1. The output shows a 'Parallel Log Analysis Result' with various statistics: Processes used: 1, DoS patterns detected: 1167, Reconnaissance patterns detected: 1759, Backdoor patterns detected: 534, Suspicious connections detected: 3460, Global checksum: 29833185496, Execution Time: 4.539708 seconds. It also indicates a 'Distributed Attack Detected (flag sum = 1)' and that 'Detailed logs saved in execution\_log.txt'.

```
(eman@kali)-[~/project]
$ mpirun -np 1 --hostfile hosts ./Q1

===== Parallel Log Analysis Result =====

Processes used:                1

DoS patterns detected:         1167
Reconnaissance patterns detected: 1759
Backdoor patterns detected:     534

Suspicious connections detected: 3460
Global checksum:               29833185496
Execution Time:                4.539708 seconds

[!] Distributed Attack Detected (flag sum = 1)

Detailed logs saved in execution_log.txt
```

Figure 1: 1 process

```
(eman@kali)-[~/project]
$ mpirun -np 2 --hostfile hosts ./Q1

===== Parallel Log Analysis Result =====

Processes used:                2

DoS patterns detected:         1167
Reconnaissance patterns detected: 1759
Backdoor patterns detected:    534

Suspicious connections detected: 3460
Global checksum:               29833185496
Execution Time:                3.105994 seconds

[!] Distributed Attack Detected (flag sum = 2)

Detailed logs saved in execution_log.txt
```

Figure 2: 2 Processes

```
(eman@kali)-[~/project]
$ mpirun -np 4 --hostfile hosts ./Q1

===== Parallel Log Analysis Result =====

Processes used:                4

DoS patterns detected:         1167
Reconnaissance patterns detected: 1759
Backdoor patterns detected:    534

Suspicious connections detected: 3460
Global checksum:               29833185496
Execution Time:                2.536262 seconds

[!] Distributed Attack Detected (flag sum = 4)

Detailed logs saved in execution_log.txt
```

Figure 3: 4 Processes

```
(eman@kali)-[~/project]
$ mpirun -np 8 --hostfile hosts ./Q1

===== Parallel Log Analysis Result =====

Processes used:                8

DoS patterns detected:         1167
Reconnaissance patterns detected: 1759
Backdoor patterns detected:    534

Suspicious connections detected: 3460
Global checksum:               29833185496
Execution Time:                14.720785 seconds

[!] Distributed Attack Detected (flag sum = 8)

Detailed logs saved in execution_log.txt
```

Figure 4: 8 Processes

## 4. Q2 : Cross-Process Correlation and Error Checking

---

Q2 extends Q1 by adding three additional capabilities required by the project brief: distributed attack detection using cross-process IP correlation, destination port scanning detection, and checksum-based validation to confirm all log segments were processed correctly.

### 4.1 Distributed Attack Detection

After all files are processed, each process has independently accumulated a list of unique suspicious source IPs and destination ports seen in its assigned chunks. The key insight is that an IP or port is only evidence of a coordinated distributed attack if it appears across multiple distinct processes meaning the malicious activity spanned different portions of the dataset, which in a real deployment would indicate activity spread across time or network segments.

#### Why MPI\_Gatherv instead of MPI\_Gather

Each process has a different number of unique IPs and ports. MPI\_Gather requires an identical receive count from every sender, which fails when counts differ. MPI\_Gatherv accepts per-process recvcunts[] and displs[] arrays, allowing rank 0 to receive exactly the right amount from each sender without any buffer overrun. This was a critical correctness fix from the original code.

#### Cross-check algorithm

Rank 0 builds an ip\_owner[] mapping array that records which process contributed each IP slot in the gathered flat array. It then iterates over every unique IP, counts how many distinct processes reported it using a proc\_seen[] bitset, and flags it as a distributed attack source only if the count exceeds 1. This prevents false positives from a single process seeing the same IP across multiple batches.

### 4.2 Destination Port Scanning Detection

The project brief requires detecting patterns involving 'multiple IP addresses OR ports'. In addition to source IPs, Q2 extracts the destination port (column 3 of the dataset) for every malicious connection. The same cross-process correlation logic is applied: ports seen targeted by connections from 2+ distinct processes are flagged as distributed scan targets.

#### Results with 4 processes

- 4 source IPs confirmed as distributed attack sources (175.45.176.0 through 175.45.176.3)
- 37 destination ports confirmed as distributed scan targets
- Notable flagged ports: 80 (HTTP), 443 (HTTPS), 23 (Telnet), 21 (FTP), 53 (DNS), 139/445 (SMB), 5900 (VNC)
- All flagged ports are well-known services consistent with real reconnaissance and exploitation patterns

### 4.3 MPI Collectives Used in Q2

| Collective    | Purpose                                                                                                   |
|---------------|-----------------------------------------------------------------------------------------------------------|
| MPI_Bcast     | Broadcast batch_count per iteration; broadcast distributed_count and final IP/port lists to all processes |
| MPI_Scatter   | Distribute log chunks evenly to all processes each batch                                                  |
| MPI_Reduce    | Aggregate per-batch malicious hit count and checksum to rank 0                                            |
| MPI_Gather    | Collect per-process IP counts and port counts into arrays on rank 0                                       |
| MPI_Gatherv   | Gather variable-length IP and port arrays from each process using recvcunts[] and displs[]                |
| MPI_Allreduce | All ranks raise a distributed attack flag after global counts are broadcast                               |
| MPI_Wtime     | Measure total wall-clock execution time                                                                   |

### 4.4 Checksum Validation

Each process computes a byte-sum checksum over every raw log row it processes. MPI\_Reduce aggregates these per-batch into a global checksum on rank 0. The final global checksum of 29,833,185,496 matches the serial.c output exactly across all process counts (1, 2, 4, and 8), confirming that no log segments were skipped, double-counted, or corrupted during parallel execution. This serves as the integrity guarantee required by the project brief.

### 4.5 Detailed Execution Log

Q2 produces q2\_execution\_log.txt which records: per-batch malicious hit counts and checksum contributions; per-process IP and port counts after the gather phase; the raw IP list contributed by each process; the full cross-process correlation verdict for every evaluated IP and port; checksum validation section; and the final deduplicated broadcast lists.

```
(eman@kali)-[~/project]
$ mpirun -np 1 --hostfile hosts ./newQ2

===== Q2: Cross-Process Correlation Results =====

Total Malicious Activities      : 3460
Global Checksum                 : 29833185496

Distributed Source IPs (seen in 2+ processes) : 0
Distributed Dest Ports (seen in 2+ processes) : 0

No distributed source IPs found.

No distributed destination ports found.

Processes used : 1
Execution Time : 4.933929 seconds

[OK] No distributed attack pattern detected.

Detailed logs saved in q2_execution_log.txt
```

Figure 5: 1 Process

```
(eman@kali)-[~/project]
$ mpirun -np 2 --hostfile hosts ./newQ2

===== Q2: Cross-Process Correlation Results =====

Total Malicious Activities      : 3460
Global Checksum                 : 29833185496

Distributed Source IPs (seen in 2+ processes) : 4
Distributed Dest Ports (seen in 2+ processes) : 32

Suspicious Source IPs:
[1] 175.45.176.0
[2] 175.45.176.3
[3] 175.45.176.1
[4] 175.45.176.2

Suspicious Destination Ports:
[1] port 80
[2] port 111
[3] port 502
[4] port 162
[5] port 110
[6] port 161
[7] port 21
[8] port 5002
[9] port 25
[10] port 0
[11] port 8080
[12] port 445
[13] port 143
[14] port 554
[15] port 443
[16] port 53
[17] port 4433
[18] port 6667
[19] port 548
[20] port 67
[21] port 23
[22] port 69
[23] port 5060
[24] port 389
[25] port 5900
[26] port 123
[27] port 5555
[28] port 139
[29] port 2000
[30] port 6905
[31] port 9999
[32] port 88

Processes used : 2
Execution Time : 3.312570 seconds

[!] Distributed Attack Confirmed across multiple processes.

Detailed logs saved in q2_execution_log.txt
```

*Figure 6: 2 Processes*

```
(eman@kali)-[~/project]
$ mpirun -np 4 --hostfile hosts ./newQ2

===== Q2: Cross-Process Correlation Results =====

Total Malicious Activities      : 3460
Global Checksum                 : 29833185496

Distributed Source IPs (seen in 2+ processes) : 4
Distributed Dest Ports (seen in 2+ processes) : 37

Suspicious Source IPs:
[1] 175.45.176.0
[2] 175.45.176.3
[3] 175.45.176.1
[4] 175.45.176.2

Suspicious Destination Ports:
[1] port 80
[2] port 111
[3] port 502
[4] port 162
[5] port 110
[6] port 161
[7] port 21
[8] port 5002
[9] port 25
[10] port 0
[11] port 53
[12] port 4433
[13] port 445
[14] port 6667
[15] port 548
[16] port 2140
[17] port 67
[18] port 143
[19] port 389
[20] port 5060
[21] port 554
[22] port 8080
[23] port 139
[24] port 9999
[25] port 88
[26] port 443
[27] port 23
[28] port 69
[29] port 5900
[30] port 123
[31] port 5555
[32] port 2000
[33] port 6905
[34] port 631
[35] port 4569
[36] port 1024
[37] port 1478

Processes used : 4
Execution Time : 2.742136 seconds

[!] Distributed Attack Confirmed across multiple processes.

Detailed logs saved in q2_execution_log.txt
```

Figure 7: 4 Processes

```
(eman@kali)-[~/project]
$ mpirun -np 8 --hostfile hosts ./newQ2

===== Q2: Cross-Process Correlation Results =====

Total Malicious Activities      : 3460
Global Checksum                 : 29833185496

Distributed Source IPs (seen in 2+ processes) : 4
Distributed Dest Ports (seen in 2+ processes) : 37

Suspicious Source IPs:
[1] 175.45.176.0
[2] 175.45.176.3
[3] 175.45.176.1
[4] 175.45.176.2

Suspicious Destination Ports:
[1] port 80
[2] port 111
[3] port 502
[4] port 162
[5] port 53
[6] port 4433
[7] port 0
[8] port 445
[9] port 161
[10] port 6667
[11] port 67
[12] port 25
[13] port 110
[14] port 389
[15] port 5060
[16] port 143
[17] port 21
[18] port 139
[19] port 5002
[20] port 548
[21] port 2140
[22] port 554
[23] port 8080
[24] port 9999
[25] port 88
[26] port 23
[27] port 5900
[28] port 5555
[29] port 2000
[30] port 443
[31] port 69
[32] port 123
[33] port 6905
[34] port 631
[35] port 4569
[36] port 1024
[37] port 1478

Processes used : 8
Execution Time : 13.727154 seconds

[!] Distributed Attack Confirmed across multiple processes.
```

Figure 8: 8 Processes

```

(eman@kali)-[~/project]
$ ./serial

===== SERIAL LOG ANALYSIS =====

DoS patterns detected:          1167
Reconnaissance patterns detected: 1759
Backdoor patterns detected:     534

Total suspicious activities:    3460
Global checksum:                29833185496
Execution Time:                 3.704266 seconds

```

Figure 9: Serial Log Analysis

## 5. Q3 : Performance Analysis

### 5.1 Experimental Setup

- Two-machine Kali Linux cluster connected over a local network
- Master machine: rank 0 reads all data, coordinates scatter/gather, writes output
- Slave machine: participates in scatter/gather/reduce collectives
- Dataset: all 4 UNSW-NB15 CSV files (~700MB total), accessed over NFS shared mount
- MPI implementation: MPICH
- Timing: MPI\_Wtime() measuring total wall-clock time from init to final output

### 5.2 Q1 Performance Results

| Mode              | Processes | Time (s) | Speedup vs Serial | Parallel Efficiency |
|-------------------|-----------|----------|-------------------|---------------------|
| Serial (baseline) |           | 3.704    | 1.00x             | 100%                |
| MPI               | 1         | 4.540    | 0.82x             | 82%                 |
| MPI               | 2         | 3.106    | 1.19x             | 60%                 |
| MPI               | 4         | 2.536    | 1.46x             | 37%                 |
| MPI               | 8         | 14.721   | 0.25x             | 3%                  |

### 5.3 Q2 Performance Results

| Processes | Time (s) | vs Serial | Dist. IPs    | Dist. Ports  |
|-----------|----------|-----------|--------------|--------------|
| 1         | 4.934    | 0.75x     | 0 (expected) | 0 (expected) |
| 2         | 3.313    | 1.12x     | 4            | 32           |
| 4         | 2.742    | 1.35x     | 4            | 37           |
| 8         | 13.727   | 0.27x     | 4            | 37           |

## 5.4 Analysis of Results

### Why MPI with 1 process is slower than serial

Even with a single process, MPI initialises its full runtime, establishes communicators, and executes MPI\_Bcast, MPI\_Scatter, MPI\_Gather, MPI\_Reduce, and MPI\_Allreduce on every batch. With 140 batches across 4 files, that is hundreds of collective operations with zero parallelism benefit. The overhead adds approximately 0.8 seconds compared to the bare serial implementation which has none of this infrastructure.

### Why 4 processes is the optimal point

The two machines together provide approximately 4 physical CPU cores available for parallel computation. At 4 processes, the speedup from splitting the computational work outweighs the communication overhead introduced by the MPI collectives. Each process handles roughly 5,000 rows per batch instead of 20,000, reducing per-process compute time by approximately 4x while paying only a modest communication cost.

### Why 8 processes causes severe degradation

Going from 4 to 8 processes causes two compounding problems. First, more processes compete for the same physical cores, introducing significant context-switching overhead. Second, every MPI collective now requires 8 participants communicating over the network link between the two machines. With 140 batches, MPI\_Scatter alone generates 140 network transmissions per run. At 8 processes, MPI\_Gatherv for IP and port arrays also becomes expensive as it must handle 8 variable-length contributions. The network becomes the bottleneck well before the CPU does.

### Communication overhead boundary data exchanges

The brief specifically asks to discuss communication overhead from boundary data exchanges. In this implementation, the primary overhead sources are: MPI\_Scatter of chunk \* MAX\_LINE bytes per process per batch (20,000 rows \* 2,048 bytes = 40MB per batch scatter across all processes); MPI\_Gather collecting 3 integer arrays of size 'processes' per batch; MPI\_Reduce of a long long checksum per batch; and MPI\_Gatherv collecting variable-length IP and port arrays at the end. The batch size of 20,000 was chosen to balance memory usage with scatter efficiency smaller batches would reduce memory pressure but increase the number of collective operations and therefore communication overhead.

### Load imbalance observation

The per-process breakdown logs reveal significant load imbalance. In batch 5 of file 1, processes 0 and 1 each received over 400 malicious rows while processes 2 and 3 received under 50. This is because MPI\_Scatter distributes rows sequentially the attack traffic in the UNSW-NB15 dataset is clustered in time, so whichever process receives the rows containing the DoS cluster does substantially more string matching work. A dynamic load-balancing approach such as MPI\_Ssend-based work stealing could address this but would significantly increase implementation complexity.

### Detection coverage vs process count (Q2)

An interesting Q2 observation is that running with 2 processes detects only 32 distributed ports compared to 37 with 4 or 8 processes. With fewer processes each receives larger data chunks, so ports that would be split across process boundaries in a 4-process run end up in the same process's chunk and are not cross-flagged. This means detection coverage improves with more processes up to the degradation point, demonstrating a trade-off between performance and detection sensitivity.

## 5.5 Speedup Calculation Amdahl's Law Context

The theoretical maximum speedup is limited by the serial fraction of the program. In this implementation, rank 0 must sequentially read each batch from disk before distributing it this serial I/O component is not parallelised.

Amdahl's Law predicts that if even 10% of the program is serial, the maximum speedup with infinite processes is only 10x. In practice the observed speedup of 1.46x at 4 processes is reasonable given the I/O bottleneck and the network communication overhead of a two-machine cluster.

## 6. PDC Concepts Applied

---

### 6.1 Message Passing Paradigm

The entire project uses the message-passing model of parallel computing where processes have separate private memory spaces and communicate explicitly by sending and receiving messages. This contrasts with shared-memory parallelism (OpenMP/threads) where all threads access a common address space. MPI's message-passing model is appropriate for distributed systems where processes run on separate physical machines, as is the case with our master-slave cluster setup.

### 6.2 Master-Worker (Manager-Worker) Pattern

The implementation uses the master-worker pattern. Rank 0 acts as the master: it performs all I/O (reading CSV files), coordinates work distribution (MPI\_Scatter), collects results (MPI\_Gather, MPI\_Reduce), performs the cross-process correlation analysis, and produces the final output. All other ranks act as workers: they receive data, perform computation independently, and report results. This pattern is natural for I/O-bound problems where a single process must serialise disk access.

### 6.3 Data Decomposition

The parallelisation strategy is based on data decomposition – dividing the input dataset across processes. Each process receives a contiguous chunk of rows (a horizontal partition of the data) and processes them independently. This is sometimes called domain decomposition in scientific computing. The batch-and-scatter approach handles datasets larger than memory by processing one batch at a time, making it a streaming data decomposition pattern.

### 6.4 Collective Communication Operations

The project makes extensive use of MPI collective operations, each serving a specific architectural purpose:

- **MPI\_Bcast** – one-to-all broadcast. Used to disseminate control information (batch\_count, global\_total) and final results (IP/port lists) from rank 0 to all ranks simultaneously. More efficient than individual point-to-point sends.
- **MPI\_Scatter** – one-to-all personalised scatter. Distributes different data to each process from rank 0. The fundamental work distribution primitive in this project.
- **MPI\_Gather** – all-to-one collection. Collects per-process results (attack counts, IP counts) into an array on rank 0, enabling per-process logging and subsequent aggregation.
- **MPI\_Gatherv** – variable-length all-to-one collection. Extension of MPI\_Gather that accepts different counts from each sender via recvcunts[] and displs[] arrays. Essential for collecting IP and port lists of varying size.
- **MPI\_Reduce** – all-to-one reduction with operation. Combines per-process values (checksums, hit counts) using a specified operator (MPI\_SUM) into a single value on rank 0. More efficient than gather-then-sum.
- **MPI\_Allreduce** – all-to-all reduction. Like Reduce but the result is available to all processes. Used for the distributed attack flag so every rank can confirm the detection without a subsequent broadcast.

## 6.5 Synchronization and Correctness

MPI collective operations are inherently synchronizing – all processes in the communicator must call the collective before any can proceed. This provides implicit barrier synchronisation at each collective point, ensuring that no process begins the next batch before all processes have completed the current one. The checksum system provides an additional correctness guarantee: by comparing the parallel global checksum against the serial result, we can detect any data corruption, skipped rows, or double-counting introduced by the parallel execution.

## 6.6 Scalability and Amdahl's Law

The performance results demonstrate Amdahl's Law in practice. The serial fraction of the program – primarily rank 0's disk I/O and the end-of-run cross-check computation – limits the maximum achievable speedup regardless of how many processes are added. Beyond the optimal process count (4 in our setup), communication overhead dominates and performance degrades. This is the fundamental scalability wall in distributed parallel computing.

## 6.7 Load Balancing

The project uses static load balancing – each process receives exactly the same number of rows (ceiling division ensures at most one extra row for boundary batches). This is simple and requires no runtime coordination, but it does not account for variation in per-row processing time. As observed in the execution logs, rows containing attack traffic require more string matching operations than clean rows, creating dynamic load imbalance even with perfectly balanced row distribution.

## 6.8 Latency vs Throughput Trade-offs

The batch size of 20,000 rows represents a latency-throughput trade-off. Larger batches amortise the fixed cost of each MPI\_Scatter/Gather round over more rows (higher throughput) but increase per-batch latency and memory usage. Smaller batches reduce latency but increase the number of collective operations and therefore communication overhead. The chosen batch size was found to provide a good balance for this dataset size and cluster configuration.

# 7. Conclusions

---

This project successfully implemented a parallel network intrusion detection system using MPI in C, demonstrating all required PDC concepts including data decomposition, collective communication, master-worker coordination, and distributed pattern detection.

The parallel implementation achieved a peak speedup of 1.46x over the serial baseline at 4 processes, correctly detecting 3,460 malicious activities (1,167 DoS, 1,759 Reconnaissance, 534 Backdoor) with a verified global checksum of 29,833,185,496. Q2 further identified 4 distributed attack source IPs and 37 destination ports targeted by coordinated cross-process scanning activity.

The 8-process degradation clearly demonstrates the importance of matching parallelism to available physical resources and network capacity. For this dataset and cluster configuration, 4 processes represents the optimal operating point where computational parallelism benefits outweigh MPI communication overhead.

The most significant technical contribution of this project beyond the baseline implementation is the set of correctness fixes applied: the memset padding fix eliminating the checksum mismatch bug, the MPI\_Allreduce correctness fix ensuring all ranks genuinely participate in collective decisions, the replacement of MPI\_Gather with MPI\_Gatherv for variable-length IP/port collection, and the cross-process owner-map algorithm preventing false-positive distributed attack detection.